Project 3 — Hangman with Categories

Python · Functions and modules (OOP optional at Level 4) · Group of 3 · Standard Library Only

Project Goal

Build a command-line Hangman game where the player chooses a category (Animals, Food, Countries), guesses a hidden word letter by letter, and sees ASCII art of the hangman as lives run out, with scores and statistics saved between sessions.

Requirements

1. Let the player choose a category.
2. Pick a random word from that category and display it as blanks.
3. Accept letter guesses, reveal correct letters, and track wrong guesses.
4. End the game with a win or lose message and offer to play again.

Example Input:

Choose a category: 1 (Animals)

Guess a letter: e

Expected Output:

```
Category: Animals

   +---+
   |   |
   O   |
       |
       |
       |
=========

Word:  _ _ E _ _ A _ _
Wrong guesses: x, q   (Lives left: 4)
Guessed so far: a, e, q, x
```

Project Levels

Level 1 — Basic (Medium)

5. Store a small list of words in the program and pick one at random.
6. Show the word as underscores, reveal letters as the player guesses correctly.
7. Give the player 6 lives, reduce one per wrong guess, and print a win or lose message at the end.

Level 2 — Intermediate (Medium–Hard)

8. Add a category menu with at least three categories (Animals, Food, Countries), each with its own word list.
9. Draw ASCII art for the hangman that changes with each wrong guess (7 stages).
10. Display the letters already guessed, and do not penalise the player for repeating a letter.
11. Validate input: accept only a single alphabetic character, case-insensitive, and re-prompt on invalid input.

Level 3 — Problem Solving (Hard)

12. Load the word lists from external text files (one file per category) and handle a missing or empty file with a clear message instead of crashing.
13. Add difficulty levels (Easy, Medium, Hard) that select words by length and set the number of lives.
14. Add a hint option that reveals one unguessed letter at the cost of one life, and allow only one hint per round.
15. Keep a running score across rounds (points based on lives left and word length), and show it after each round with a play-again prompt.

Level 4 — Challenge (Hard)

16. Save a high-score table to a JSON or CSV file that persists between sessions, showing the top 5 players by name.
17. Add a two-player mode: Player 1 types a secret word (hidden from the screen) and a category hint, Player 2 guesses.
18. Allow guessing the whole word at once, with a larger penalty for a wrong full-word guess, and handle words with spaces or hyphens (e.g., "New Zealand").
19. Track and display statistics: games played, win rate, win rate per category, longest winning streak, and the most-missed letters.
20. Let players add new words or a new category from within the game menu, saving them to the word files and rejecting duplicates and invalid entries.

Suggested Role Split (Group of 3)

- **Member A — Game logic:** word selection, guess checking, lives, win/lose conditions, whole-word guesses, hints.
- **Member B — Display and input:** menus, ASCII art stages, input validation, two-player screen handling.
- **Member C — Data and persistence:** word files, scoring, high scores, statistics, adding words and categories.

Agree on how data is passed between parts first (for example, a game-state dictionary holding `word`, `guessed`, `lives`, `hint_used`), so the three parts fit together at the end.

Skills Practiced

Random selection, loops and conditionals, string handling, sets and lists, functions and modules, file reading and writing, JSON or CSV persistence, input validation, and teamwork on a shared data structure.